Lab 10 — Hands-on lab: operating the stack

Goal: on the lab 09 stack, read logs and metrics, set limits and trigger an OOM, engrave a `HEALTHCHECK` into the image (and discover why Podman loses it), harden a container, push to a private registry, clean up without breaking anything, then write a Quadlet for starting at boot.

Prerequisites — Labs 01 to 09 completed: `~/labo-docker/09/` complete, images `api:1.0`, `postgres:16-alpine`, `nginx:alpine`, `alpine`; podman-compose installed; on WSL, `systemd=true` (required for `podman stats` and for step 8).

Files provided (`files/`) - `api.container` — the Quadlet for step 8.

Step 0 — Restarting the stack

```
mkdir -p ~/labo-docker/10 && cd ~/labo-docker/09 && podman compose up -d ; sleep 12 ; cd
~/labo-docker/10
podman compose -f ~/labo-docker/09/compose.yaml ps 2>/dev/null | grep -c healthy
```

Observe `2`: `db` and `api` are `healthy`, `web` has no healthcheck.

Step 1 — Reading the logs

```
podman logs -t --tail=2 stack_api_1
podman inspect stack_api_1 --format '{{.HostConfig.LogConfig.Type}}
{{.HostConfig.LogConfig.Path}}'
curl -s localhost:18080/api/ > /dev/null ; podman logs --tail=1 stack_web_1
```

Observe two timestamped lines (`2026-…T12:58:00.068793040Z Arguments recus : (aucun)`); the `k8s-file` driver and a path `…/overlay-containers/<id>/userdata/ctr.log`; then the nginx access line `"GET /api/ HTTP/1.1" 200 …` generated by your `curl`.

Explanation. Each container has its log file, written by `conmon`. `-t` adds the timestamp, `--since`/`--tail` bound the read. That file has no maximum size by default.

Step 2 — Processes, resources, events

```
podman top stack_api_1 user pid args
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}'
podman inspect stack_api_1 --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}
pids={{.HostConfig.PidsLimit}}'
podman events --stream=false --since 5m --filter container=stack_api_1 | cut -c1-60
```

Observe `podman 1 java -jar /app/api.jar` (the image's user 1000, shown with your name); in `stats`, a few percent of CPU and `27.58MB / 32.48GB`: the "limit" is the machine's memory; `mem=0 cpus=0 pids=0`: no limit set; and the list of events `container init`, `container start`, then a `health_status` every five seconds.

Explanation. `stats` reads cgroups, `top` the processes, `inspect` the configuration, `events` the engine's journal. Without `--memory`, nothing stops this container before the whole machine.

Step 3 — Limiting, and triggering an OOM

```
podman run -d --name api-limit --memory 256m --cpus 0.5 api:1.0 ; sleep 3
podman inspect api-limit --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}'
podman stats --no-stream --format '{{.Name}} {{.MemUsage}} {{.MemPerc}}' api-limit
podman exec api-limit cat /sys/fs/cgroup/memory.max
podman rm -f -t 0 api-limit
```

Observe `mem=268435456 cpus=500000000` (bytes and nano-CPUs), `21.46MB / 268.4MB 8.00%`, and from inside the container `memory.max` = `268435456`: the limit is a cgroup file the process can read.

```
podman run --name oom --memory 32m alpine sh -c 'head -c 100m /dev/zero | tail' ; echo
"exit=$?"
podman inspect oom --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'
podman rm oom
```

Observe `Killed`, `exit=137`, then `oom=true exit=137`: `tail` wanted to keep 100 MB in memory within 32 MB, the kernel killed it.

```
podman run -d --name api-heap --memory 256m api:1.0 ; sleep 3
podman exec api-heap sh -c 'java -XX:+PrintFlagsFinal -version 2>/dev/null | grep "
MaxHeapSize"'
podman exec api-heap sh -c 'java -XX:MaxRAMPercentage=50 -XX:+PrintFlagsFinal -version
2>/dev/null | grep " MaxHeapSize"'
podman rm -f -t 0 api-heap
```

Observe `MaxHeapSize = 132120576` (126 MB: under a small limit, the JVM takes half by default, not 25 %) then `134217728` (exactly 128 MB with `MaxRAMPercentage=50`): the JVM read the container's limit and sizes its heap on it.

Explanation. `--memory` writes `memory.max` into the container's cgroup; exceeding it is not an application error but a `SIGKILL` from the kernel. For Java, the heap is set as a percentage of that limit, through `JAVA_TOOL_OPTIONS`.

Step 4 — A `HEALTHCHECK` in the image

```
printf 'FROM localhost/api:1.0\nHEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -
qO- http://localhost:8080/actuator/health || exit 1\n' > Dockerfile.health
podman build -q -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'
podman build -q --format docker -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck.Test}}'
```

Observe on the first build a warning `HEALTHCHECK is not supported for OCI image format and will be ignored. Must use` docker `format`, then `<nil>` — the instruction is gone — then `[CMD-`

`SHELL wget -qO- http://localhost:8080/actuator/health || exit 1]` after the `--format docker` build.

```
podman run -d --name api-h api:1.1 ; sleep 9
podman ps --filter name=api-h --format '{{.Names}} {{.Status}}'
podman healthcheck run api-h ; echo "exit=$?"
podman inspect api-h --format '{{.State.Health.Status}} {{len .State.Health.Log}}'
podman exec api-h sh -c 'kill 1' ; sleep 1 ; podman ps -a --filter name=api-h --format
'{{.Status}}'
podman rm -f -t 0 api-h
```

Observe `api-h Up 9 seconds (healthy)`, `exit=0`, `healthy` with two checks already in the history; and after `kill 1`, `Exited (143) 1 second ago (healthy)`: the healthcheck restarted nothing, it is an indicator — one that even keeps its last state.

Explanation. The OCI format has no field for the healthcheck; Buildah drops it with a one-line warning. With `--format docker`, the image carries its own probe, run by a systemd timer under rootless.

Step 5 — Hardening

```
podman run --rm --entrypoint id api:1.0
podman run --rm --entrypoint sh api:1.0 -c 'touch /app/x && echo "write OK in /app"'
podman run --rm --read-only --tmpfs /tmp --entrypoint sh api:1.0 -c 'touch /app/x || echo
"read-only" ; touch /tmp/x && echo "tmp OK"'
```

Observe `uid=1000(…) gid=1000(…)`: the image already runs as non-root (`USER 1000:1000`, lab 05); `touch: cannot touch '/app/x': Permission denied` (the folder belongs to root); then with `--read-only`, `Read-only file system` everywhere except in the `tmpfs`.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo "chown OK (root
with capabilities)"'
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'
podman run --rm --security-opt no-new-privileges --cap-drop ALL --read-only --tmpfs /tmp --user
1000 --entrypoint id alpine
```

Observe `chown OK` for the container's ordinary root, `chown: /etc/hostname: Operation not permitted` without capabilities, and `uid=1000 gid=0(root)` for the full hardening line — which starts without error.

Explanation. Non-root, read-only, zero capabilities, no escalation: four options, no line of code. These options have their exact equivalent in Compose and in Kubernetes.

Step 6 — A private registry

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.0 localhost:5000/api:1.0
podman push localhost:5000/api:1.0 2>&1 | tail -1 | cut -c1-120
podman push --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
curl -s localhost:5000/v2/_catalog ; echo ; curl -s localhost:5000/v2/api/tags/list ; echo
```

Observe `Error: … Get "https://localhost:5000/v2/": http: server gave HTTP response to HTTPS client`, then `Writing manifest to image destination`, and the registry API: `{"repositories":["api"]}` and `{"name":"api","tags":["1.0"]}`.

```
podman rmi localhost:5000/api:1.0
podman pull --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep 'api:1.0'
podman image inspect api:1.0 --format '{{.Digest}}'
podman rm -f -t 0 -v registry
```

Observe after the round trip, `localhost:5000/api:1.0` and `localhost/api:1.0` with the **same** `IMAGE ID`; a digest `sha256:52b82ed9…` (yours differs); and `rm -v`, since `registry:2` declares a `VOLUME` (lab 06).

Explanation. The registry stores blobs addressed by their hash; what comes back out is bit for bit what went in. TLS is required by default; `--tls-verify=false` is acceptable only for this test.

Step 7 — Measuring, then cleaning up

```
podman system df
podman images --filter dangling=true --format '{{.ID}} {{.Size}}'
podman image prune -f
podman system df | head -2
```

Observe the breakdown per category with the `RECLAIMABLE` column; the list of `<none>` images (on your machine, probably the leftovers of the `api:1.1` rebuilds); their IDs removed; and an image total that drops. Volumes and containers were not touched.

Explanation. `image prune` without `-a` only removes dangling images: it is the only `prune` to run by reflex. The others (`-a`, `container`, `volume`, `system`) are decided after reading `system df` and `volume ls --filter dangling=true`, and a volume is deleted **by name**.

Step 8 — Starting at boot with Quadlet (WSL)

This step requires systemd in your distribution (`systemd=true` in `/etc/wsl.conf`); it cannot be replayed in the sandbox this lab was written in, so the output below is described rather than quoted.

```
mkdir -p ~/.config/containers/systemd
cp <path-to-the-lab>/files/api.container ~/.config/containers/systemd/
systemctl --user daemon-reload
systemctl --user start api
systemctl --user status api --no-pager | head -5
podman ps --format '{{.Names}} {{.Status}} {{.Ports}}'
curl -s localhost:18080/actuator/health ; echo
journalctl --user -u api -n 3 --no-pager
```

Observe an `api.service` service `active (running)`, a container named `systemd-api` (prefix added by Quadlet) published on `18080`, `{"status":"UP"}`, and the API's logs in the systemd journal.

```
systemctl --user enable api
loginctl enable-linger $USER
systemctl --user stop api
podman ps -a --format '{{.Names}}' | grep -c systemd-api
```

Observe `0` after `stop`: Quadlet starts the container with `--rm`, stopping the service removes it; at the next start of the session (or of WSL, once the distribution is launched), systemd recreates it.

Explanation. The `.container` file is translated into a systemd unit; it is systemd, relaunched by the system, that relaunches Podman. `enable-linger` keeps the `--user` session alive without an open login. For the remaining labs, leave the service **stopped** (`systemctl --user disable --now api`) so that port 18080 stays free.

Clean-up

```
cd ~/labo-docker/09 && podman compose down && cd ~/labo-docker/10
podman rmi api:1.1 localhost:5000/api:1.0 docker.io/library/registry:2
rm -f Dockerfile.health
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman images --
format '{{.Repository}}:{{.Tag}}'
```

Observe no container, the `stack_pgdata` volume kept (the lab 09 database), and the stack images still there. End of the Docker part: the Kubernetes part (lab 11) reuses `api:1.0`, `nginx:alpine` and `postgres:16-alpine`.

What you must be able to state now

- `logs -t`, `top`, `stats`, `events`: the `ctr.log` file, the `java` process as UID 1000, `27 MB` used with no limit, a `health_status` every five seconds.
- `--memory 256m` = `memory.max` readable from inside the container; `32m` for 100 MB = `Killed`, `137`, `OOMKilled: true`; the JVM sizes its heap on the limit, `MaxRAMPercentage` sets the share.
- `HEALTHCHECK` lost in OCI format (`<nil>`), kept with `--format docker` (`(healthy)`), and it restarts nothing (`Exited (143)`).

- `--read-only` + `--tmpfs`, `--cap-drop ALL` (`Operation not permitted`), `no-new-privileges`, non-root: the hardened line starts.
- `push` refused without TLS, accepted with `--tls-verify=false`; same `IMAGE ID` after the round trip; `rm -v` for the registry.
- `system df` first, `image prune` (dangling) by reflex, the rest by name.
- Quadlet: `.container` → `api.service`, `enable` + `enable-linger` to survive `wsl --shutdown`.